

FROM VIBE CODER TO SOFTWARE ENGINEER

*An Apple-Oriented, Project-Based Software Engineering
Roadmap*

AmirMahdi

Frontend-Focused · AI-Assisted Development · Automation

September 2026

Learn · See · Try · Fail · Debug · Solve · Build · Review

Table of Contents

درباره این کتاب

نسخه 1.0 — سپتامبر 2026

با AI-assisted این کتاب یک دوره کوتاه مدت نیست. یک برنامه رشد چندساله است که نقطه شروع آن یک پروژه ساز تجربه عملی زیاد و مبانی مهندسی هنوز در حال شکل گیری است، و نقطه هدف آن آمادگی حرفه ای برای رقابت در است Apple موقعیت های مهندسی نرم افزار در شرکت هایی مانند

درباره این کتاب

این کتاب برای یک نیاز مشخص نوشته شده است:

Software پروژه های واقعی می سازد، طی چند سال به یک AI چگونه کسی که امروز با کمک شدید

مستقل، عمیق و حرفه ای تبدیل شود؟ Engineer

این کتاب قرار نیست فقط فهرستی از تکنولوژی ها باشد.

چرخه یادگیری کتاب

Learn See Try Fail Debug Solve Build Review

یعنی:

یاد بگیر → ببین → امتحان کن → اشتباه کن → اشکال زدایی کن → حل کن → بساز → مرور کن

در این مسیر

پروژه های واقعی حفظ می شوند و به سرمایه آموزشی تبدیل می شوند.

حذف نمی شود؛ بلکه از «جایگزین فکر» به «ابزار افزایش بهره وری مهندس» تبدیل می شود AI استفاده از

قربانی سرعت نمی‌شوند JavaScript و Computer Science مبانی

تخصص اصلی باقی می‌ماند Frontend.

Frontend Engineer به اندازه‌ای یاد گرفته می‌شوند که یک Backend، Systems و Cloud بتواند کل سیستم را بفهمد.

از روز اول همراه آموزش خواهد بود Technical English.

و ارزیابی همراه است Debugging، هر بخش با تمرین

«Apple درباره هدف»

ها، نیاز Job description. هدف نهایی این برنامه است، اما استخدام هرگز از پیش تضمین‌شدنی نیست Apple. تیم‌ها، تحصیلات، تجربه حرفه‌ای، شرایط جغرافیایی و نتیجه مصاحبه در زمان اقدام اهمیت دارند.

پس هدف قابل‌کنترل ما این است:

نیز Apple ساختن مهندسی که واقعاً برای موقعیت‌های مهندسی سطح بالا آماده باشد و بتواند برای رقابت کند.

چگونه از این کتاب استفاده کنیم؟

AI قانون 1 – اول فکر، بعد

در مرحله‌های پایه

مسئله را بخوان.

ورودی و خروجی را مشخص کن.

الگوریتم را روی کاغذ یا ذهن بنویس.

حداقل یک بار خودت تلاش کن.

کمک بگیر AI سپس از

را مالک شو AI قانون 2 – کد

:کدی نوشته است، تا زمانی که نتوانی توضیح بدهی AI اگر

چرا درست است؟

چرا این ساختار انتخاب شده؟

چيست؟ complexity

ها کدام اند؟ edge case

چگونه تست می شود؟

آن کد از نظر آموزشی «کد خودت» محسوب نمی شود

مهم تر از تقویم است Mastery – قانون 3

:عبور از یک فصل زمانی انجام می شود که بتوانی

Understand I mplement D ebug E xplain A pply

مسیر یادگیری

Programming Fundamentals

↓

JavaScript

↓

JavaScript Internals + Async

↓

TypeScript

↓

HTML/CSS + Browser



HTTP + APIs



React



Next.js + Frontend Architecture



Testing + Debugging + Security



SQL + Databases



Networking + Linux + OS



Data Structures + Algorithms



Software Engineering



Distributed Systems + Reliability



CI/CD + Cloud + Observability



AI-Assisted Engineering



Professional Engineering



Technical English



Interview Engineering



Apple-Specific Preparation



Apple-ready

نقطه شروع — PART I

فصل 1 — تو امروز دقیقاً کجایی؟

اهداف یادگیری

در پایان این فصل بتوانی:

را توضیح بدهی «software engineer» و «project builder» فرق بین

را از هم جدا کنی AI-assisted مهارت مستقل و مهارت

مزیت‌های فعلی خودت را شناسایی کنی

شکاف‌های اصلی مسیر را بشناسی

برای یادگیری، معیار واقع‌بینانه تعیین کنی

پیش‌نیاز

هیچ

وضعیت فعلی

تو از صفر شروع نمی‌کنی

تجربه‌های فعلی شامل موضوعاتی مانند

Frontend

HTML/CSS/JavaScript

API integration

Git/GitHub

GitHub Actions

Cron

Cloudflare Workers

Serverless

Telegram/Bale bots

Automation

Playwright

Deployment

JSON APIs

AI-assisted development

چند پروژه مورد استفاده واقعی

freelance تجربه

است.

مستقل یکسان نیست mastery اما این تجربه با

اصل مهم

ممکن است کسی پروژه بسیار پیچیده‌ای داشته باشد اما هنوز در نوشتن یک الگوریتم ساده بدون کمک مشکل داشته باشد.

این تناقض نیست.

پروژه‌سازی و عمق مهندسی دو محور جدا هستند

مثال

می‌تواند این پروژه را برای تو بسازد AI فرض کن یک

API → Database → Worker → Bot → Cron

کنی، خطاها را دنبال کنی و نیازهای سیستم را مشخص کنی، یک توان واقعی داری deploy اگر بتوانی آن را

توضیح بدهی AI اما هنوز باید بتوانی بدون

چیست؟ API

چگونه منتقل می‌شود؟ request

چیست؟ database transaction

چه محدودیتی دارد؟ cron

چیست؟ race condition

این‌ها لایه دوم مهارت‌اند

🖋 امتحانش کن

سه پروژه‌ای را که بیشتر به آن‌ها افتخار می‌کنی انتخاب کن

برای هر پروژه فقط این چهار سؤال را جواب بده

مشکل چه بود؟

چه چیزی ساختی؟

چه بخش‌هایی را خودم می‌فهمم؟

وابسته‌ام؟ AI کجاها هنوز به

هدف، صداقت است؛ نه نمره.

خطای رایج

«**اشتباه:**» چون پروژه کار می‌کند، پس تکنولوژی آن را بلدم

چرا مشکل است؟

ساخته شود در حالی که نویسنده configuration کتابخانه‌ها و AI-generated یک پروژه می‌تواند با ترکیبی از کد هنوز علت کارکرد همه اجزای آن را نمی‌داند.

روش بررسی

یک فایل از پروژه را باز کن و از خودت بپرس:

توضیح بدهم؟ AI آیا می‌توانم هر بخش را بدون پرسیدن از

در نظر بگیر «AI-assisted exposure» اگر نه، سطح آن بخش را

تمرین

تمرین 1

سه مهارت مستقل خودت را نام ببر.

تمرین 2

هستند مشخص کن AI-assisted سه مهارتی که فعلاً بیشتر

چالش

توضیح بده چرا پروژه پیچیده الزاماً به معنی مهارت مستقل پیچیده نیست.

Quiz

شده الزاماً نشان می‌دهد که تمام کد آن را مستقل نوشته‌ای؟ deploy آیا پروژه

چیست؟ Project Experience و Independent Mastery تفاوت

را نباید حذف کرد؟ AI-assisted experience چرا

خلاصه

مزیت تو:

توان ساختن و اتصال سیستم‌ها

شکاف اصلی:

عمق، استقلال و مبانی مهندسی

✓ Checklist

- ☐ را می‌فهمم mastery تفاوت تجربه پروژه‌ای و
- ☐ جدا می‌کنم AI-assisted مهارت مستقل خودم را از مهارت
- ☐ می‌دانم هدف اصلی کتاب چیست

مسیر پیشرفت

- Programming ①
 - Project Building ✓
 - Automation ✓
 - AI-assisted Development ✓
 - Independent Engineering ①
-

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

خلاصه فصل

و مهارت مستقل برنامه‌نویسی هنوز در حال AI نقطه شروع ما ترکیبی از تجربه عملی بالا در ساخت پروژه با رشد است. هدف این کتاب این است که این دو را به‌مرور هم‌سطح کند.

فصل 2 – سیستم یادگیری مهندس

اهداف یادگیری

یک مسئله را قبل از کدنویسی تحلیل کنی.

به شکل کنترل‌شده استفاده کنی از AI.

یک چرخه یادگیری مستقل داشته باشی.

خطا را بخشی از یادگیری بدانی.

پیش‌نیاز

فصل 1.

چرخه استاندارد

Understand

↓

Plan

↓

Implement

↓

Run

↓

Observe

↓

Debug

↓

Verify

↓

Explain

این چرخه در تمام کتاب تکرار می‌شود.

چرا مهم است؟

«یک مهندس حرفه‌ای فقط «کد تولید نمی‌کند».

او باید مطمئن شود:

کدی که تولید شد، همان چیزی است که باید تولید می‌شد.

امتحانش کن

یک برنامه کوتاه را قبل از اجرا پیش‌بینی کن.

مثلاً:

```
let x = 10;
```

```
x = x + 5;
```

```
console.log(x);
```

قبل از اجرا جواب بده.

بعد اجرا کن.

هدف، عادت پیش‌بینی است.

خطای رایج

Vibe debugging:

جایگزینی کد → امیدوارم درست شده باشد → AI خطا → کپی خطا در

نسخه مهندسی:

خطا

→ reproduce

→ inspect

→ hypothesis

→ minimal fix

→ test

Mini Project — Debug Notebook

بِساز Markdown یک فایل:

Problem:

Expected:

Actual:

Evidence:

Hypothesis:

Fix:

Verification:

از این فایل در کل مسیر استفاده کن.

Checklist

☐ قبل از اجرا پیش‌بینی می‌کنم.

☐ دارم hypothesis یک AI قبل از

☐ را توضیح بدهم fix می‌توانم دلیل

خلاصه فصل

این فصل یک لایه از مهارت «سیستم یادگیری مهندس» را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART II — PROGRAMMING FOUNDATION

فصل 3 — JavaScript Fundamentals

اهداف یادگیری

variable و data type را بفهمی.

بنویسی loop شرط و

array و object را پردازش کنی

function بسازی.

حل کنی AI مسئله‌های ساده را بدون

پیش‌نیاز

فصول 1 و 2

Variables: مفهوم

```
let age = 17;
```

```
const name = "Amir";
```

شود assign ی است که قرار نیست دوباره binding برای const برای مقدار قابل تغییر و

چرا مهم است؟

تکیه می‌کنند state تمام الگوریتم‌های بعدی بر نگهداری

مثال

```
let count = 0;
```

```
count = count + 1;
```

```
console.log(count);
```

توضیح

count = 0 مقدار اولیه است.

count + 1 مقدار جدید را محاسبه می‌کند.

assignment مقدار جدید را نگه می‌دارد.

Conditions

```

if (age >= 18) {
  console.log("Adult");
}

```

Loops

```

for (let i = 0; i < numbers.length; i++) {
  console.log(numbers[i]);
}

```

Arrays

باید تفاوت این دو را حفظ کنی:

i = index

numbers[i] = value at that index

Objects

```

let user = {
  name: "Amir",
  age: 17
};

console.log(user.name);

```

Functions

```
function add(a, b) {  
    return a + b;  
}
```

```
console.log(add(5, 3));
```

امتحانش کن

در مثال بالا:

را عوض نکن a و b.

را تغییر بده arguments مقدار

قبل از اجرا خروجی را حدس بزن.

خطاهای رایج

خطا 1

```
if (numbers[i] = 25) {  
}
```

comparison به جای assignment: مشکل.

اصلاح:

```
if (numbers[i] === 25) {  
}
```

خطا 2

```
for (let i = 0; i < numbers.length;) {  
}
```

ممکن است متوقف نشود loop، تغییر نکند i اگر.

خطا 3

```
console.log(number[i]);
```

است numbers در حالی که متغیر

تمرین‌ها

● تمرین ساده

بزرگ‌ترین عدد یک آرایه را پیدا کن

● تمرین متوسط

ها بشمار object تعداد عناصر بزرگ‌تر یا مساوی 18 را در آرایه‌ای از

● چالش

یک عدد را برگرداند و اگر پیدا نشد -1 بدهد occurrence اولین index تابعی بنویس که

🧠 راهنمای حل

یک مقدار اولیه نگه دار

آرایه را پیمایش کن

شرط را اعمال کن

مناسب نگه دار state نتیجه را در

را فراموش نکن not found حالت

🔧 Mini Project — Array Analyzer

ورودی:

```
let numbers = [12, 7, 25, 3, 19];
```

خروجی مورد انتظار

sum

min

max

average

targetIndex

چالش

استفاده نکن max/min یا built-in sorting هیچ

✓ Checklist

Variables

☐ Conditions

☐ Loops

☐ Arrays

☐ Objects

☐ Functions

☐ Return

☐ Search

☐ Min/Max

☐ Count

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «JavaScript Fundamentals» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 4 – Array و Object Thinking

اهداف یادگیری

را با زبان خودت توضیح بدهی «Array و Object Thinking» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Array و Object Thinking» یک مثال واقعی مرتبط با

را تشخیص بدهی «Array و Object Thinking» های مرتبط با trade-off خطاهای رایج و

پیش نیاز

ساده JavaScript و توانایی نوشتن Programming Foundation فصل‌های قبلی بخش

اهداف

ها را بخوانی object آرایه‌ای از

را درست استخراج کنی property

کنی count و filter داده را

حذف و جابه‌جایی را بفهمی

مثال

```
let users = [
  { name: "Amir", age: 17 },
  { name: "Ali", age: 20 },
  { name: "Sara", age: 16 }
];

for (let i = 0; i < users.length; i++) {
  if (users[i].age >= 18) {
    console.log(users[i].name);
  }
}
```

فکر کردن

users[i] → object

users[i].age → property

users[i].name → output

امتحانش کن

تغییر بده و قبل از اجرا خروجی را پیش‌بینی کن $age < 18$ را به $age \geq 18$

حذف دستی

splice برای حذف بدون

```
for (let j = i; j < numbers.length - 1; j++) {
  numbers[j] = numbers[j + 1];
}
```

```
numbers.length--;
```

این الگوریتم از ایده «شیفت عناصر بعدی به چپ» استفاده می‌کند.

خطای رایج

```
numbers[j] = numbers[j++];
```

به جای:

```
numbers[j] = numbers[j + 1];
```

قرار نمی‌دهیم access را داخل counter دوم، مقدار بعدی را می‌خواهیم؛ افزایش expression در

Challenge

هایی را که نامشان با حرف مشخصی شروع می‌شود پیدا کن user تمام

Mini Project — User Filter

داده

```
let users = [
  { name: "Amir", age: 17 },
  { name: "Ali", age: 20 },
  { name: "Sara", age: 16 },
  { name: "Reza", age: 22 }
];
```

قابلیت‌ها:

تعداد بزرگسالان

تعداد افراد زیر 18

با نام user پیدا کردن

index پیدا کردن

جمع سن‌ها

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Object Thinking و Array» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

1 سؤال

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

2 سؤال

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

3 سؤال

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 5 — Functions: پارامتر، Return و Scope

اهداف یادگیری

را با زبان خودت توضیح بدهی «Functions: پارامتر، Return و Scope» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Functions: پارامتر، Return و Scope» یک مثال واقعی مرتبط با

را تشخیص بدهی «Functions: پارامتر، Return و Scope» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

ساده JavaScript و توانایی نوشتن Programming Foundation فصل‌های قبلی بخش

اهداف

را تشخیص دهی argument و parameter

return را درست استفاده کنی.

function reusable بسازی.

local variable را بفهمی.

مثال

```
function sumArray(numbers) {
  let sum = 0;

  for (let i = 0; i < numbers.length; i++) {
    sum += numbers[i];
  }

  return sum;
}
```

استفاده:

```
let numbers = [5, 10, 3];

console.log(sumArray(numbers));
```

مهم است؟ return چرا

نتیجه را نمایش می‌دهد console.log.

خارج می‌کند تا جای دیگری استفاده شود function نتیجه را از return.

امتحان کن

return sum به جای:

```
return 0;
```

بگذار و قبل از اجرا پیش‌بینی کن.

خطای رایج

```
function sumArray(numbers) {  
  for (...) {  
    return sum + numbers[i];  
  }  
}
```

تمام شود iteration در اولین function باعث می‌شود loop داخل return

تمرین‌ها



تابع findMax(numbers).



تابع countAdults(users).



کند و آرایه خالی را هم مدیریت کند return آرایه را average تابعی بنویس که

Mini Project — Calculator Functions

توابع:

add

subtract

multiply

divide

average

max

min

بعد یک تابع

calculate(operation, a, b)

اضافه کن.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار «Scope و Return، پارامتر: Functions» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، یادگیری، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 6 — JavaScript Internals

اهداف یادگیری

را با زبان خودت توضیح بدهی «JavaScript Internals» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «JavaScript Internals» یک مثال واقعی مرتبط با

را تشخیص بدهی «JavaScript Internals» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

ساده JavaScript و توانایی نوشتن Programming Foundation فصل‌های قبلی بخش

اهداف

را در سطح مفهومی بفهمی execution context

را بفهمی lexical environment و scope

را درک کنی closure

را توضیح بدهی call stack.

را بشناسی prototypes و references.

Call Stack

قرار می‌گیرد call stack صدا زده می‌شود، اجرای آن روی function وقتی.

مثال:

```
function first() {
  second();
}
```

```
function second() {
  console.log("Hello");
}
```

```
first();
```

ذهنی:

```
first()
```

↓

```
second()
```

↓

```
console.log()
```

Closure

ببیرونی خود دسترسی را حفظ می‌کند lexical به محیط function حالتی است که Closure.

مثال:

```
function counter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}
```

```
const next = counter();
```

```
console.log(next());
```

```
console.log(next());
```

امتحانش کن

در اجرای دوم مقدار متفاوتی می‌دهد next() ببین چرا.

خطای رایج

هستند global تصور اینکه همه متغیرها.

تمرین

بساز که بتواند function counter یک

increment

decrement

read

انجام دهد.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «JavaScript Internals» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 7 — Asynchronous JavaScript

اهداف یادگیری

را با زبان خودت توضیح بدهی «Asynchronous JavaScript» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Asynchronous JavaScript» یک مثال واقعی مرتبط با

را تشخیص بدهی «Asynchronous JavaScript» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

ساده JavaScript و توانایی نوشتن Programming Foundation فصل‌های قبلی بخش

اهداف

را فرق بگذاری sync و async

را بفهمی Promise

را توضیح بدهی async/await

را در سطح کاربردی بفهمی event loop

را بشناسی race condition

مفهوم

Call Stack

↓

Async runtime

↓

Queues

↓

Event Loop

↓

Call Stack

Promise

```
fetch("/api/data")
```

```
.then(response => response.json())
```

```
.then(data => console.log(data))
```

```
.catch(error => console.error(error));
```

با async/await:

```
async function loadData() {
```

```
  try {
```

```
    const response = await fetch("/api/data");
```

```
    const data = await response.json();
```

```
    return data;
```

```
  } catch (error) {
```

```
    console.error(error);
```

```
  }
```

```
}
```



امتحان کن

عادی بنویس و ترتیب خروجی را قبل از اجرا پیش‌بینی کن console.log کوتاه و یک delay با setTimeout یک

خطای رایج

await فراموش کردن

```
const data = response.json();
```

داریم Promise اینجا معمولاً هنوز

خطای دیگر

دقیقاً به ترتیب نوشته شدن تمام می شود async operation فرض اینکه هر

تمرین

فرضی را به صورت همزمان اجرا کن API دو

Challenge

رفتار در صورت شکست یکی از درخواست ها را طراحی کن



Mini Project — API Dashboard

را بگیر endpoint داده های چند

weather

market

news

اگر یکی از منابع شکست خورد، کل خروجی را از بین ببر

خلاصه فصل

را به مسیر مهندسی اضافه می کند. معیار یادگیری، «Asynchronous JavaScript» این فصل یک لایه از مهارت

و به کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده سازی



Checklist

☐ مفهوم اصلی فصل را می توانم توضیح بدهم.

☐ می توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.

می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم □

می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم □

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

TypeScript – فصل 8

اهداف یادگیری

را با زبان خودت توضیح بدهی «TypeScript» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «TypeScript» یک مثال واقعی مرتبط با

را تشخیص بدهی «TypeScript» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

ساده JavaScript و توانایی نوشتن Programming Foundation فصل‌های قبلی بخش

اهداف

را برای جلوگیری از خطا استفاده کنی type

را فرق بگذاری type alias و interface

را بفهمی narrowing و union

را در سطح کاربردی استفاده کنی generics

بسازی API contract

مثال

```
interface User {  
  name: string;  
  age: number;  
}
```

Union

```
let result: string | null;
```

Narrowing

```
if (typeof value === "string") {
  console.log(value.toUpperCase());
}
```

Generic

```
function identity<T>(value: T): T {
  return value;
}
```

🔧 امتحانش کن

طراحی کن که response API برای یک type یک

success

data

error

داشته باشد.

🐛 خطای رایج

برای خاموش کردن مشکل به جای حل آن any استفاده از

🧩 Mini Project

مدل کن type-safe را Market API های پروژه response API

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «TypeScript» این فصل یک لایه از مهارت و به کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

✅ Checklist

☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART III — COMPUTER SCIENCE CORE

فصل 9 — Computational Thinking

اهداف یادگیری

را با زبان خودت توضیح بدهی «Computational Thinking» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Computational Thinking» یک مثال واقعی مرتبط با

را تشخیص بدهی «Computational Thinking» های مرتبط با trade-offهای خطاهای رایج و

پیش‌نیاز

مفاهیم پایه برنامه‌نویسی و توانایی خواندن و نوشتن کد ساده

اهداف

کنی decomposition مسئله را

را پیدا کنی constraints

را مشخص کنی edge case

را قبل از کد بررسی کنی correctness

الگوی تحلیل

Problem

Input

Output

Constraints

Edge Cases

Algorithm

Correctness

Complexity

مثال

«یک عدد را پیدا کن index اولین»

Input:

array + target

Output:

یا 1- index

Constraint:

آرایه ممکن است بزرگ باشد

امتحانش کن

قبل از کدنویسی مشخص کن اگر

آرایه خالی باشد

چند بار وجود داشته باشد target

اصلاً نباشد target

چه اتفاقی می افتد

Challenge

حل کن pseudocode از آرایه را ابتدا فقط با occurrence مسئله «حذف اولین»

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Computational Thinking» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

Big-O – فصل 10

اهداف یادگیری

را با زبان خودت توضیح بدهی «Big-O» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Big-O» یک مثال واقعی مرتبط با

را تشخیص بدهی «Big-O» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

مفاهیم پایه برنامه‌نویسی و توانایی خواندن و نوشتن کد ساده

اهداف

را بفهمی complexity

را جدا کنی space و time

ها را تحلیل کنی trade-off

نمونه

```
for (let i = 0; i < n; i++) {
  console.log(numbers[i]);
}
```

تقریباً:

$O(n)$

تو در تو loop دو

$O(n^2)$

Binary search:

$O(\log n)$

چرا مهم است؟

دو کد ممکن است هر دو «درست» باشند اما با بزرگ شدن ورودی رفتار کاملاً متفاوتی داشته باشند.

امتحان کن

را قبل از پاسخ نهایی حدس بزن complexity برای هر کد این فصل.

خطای رایج

شمردن خطوط کد به جای رشد عملیات.

تمرین

این الگوها را مشخص کن complexity

یک lookup

یک loop

مستقل loop دو

تو در تو loop دو

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «Big-O» این فصل یک لایه از مهارت و به کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

✓ Checklist

☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

11 فصل – Data Structures

اهداف یادگیری

را با زبان خودت توضیح بدهی «Data Structures» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Data Structures» یک مثال واقعی مرتبط با

را تشخیص بدهی «Data Structures» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

مفاهیم پایه برنامه‌نویسی و توانایی خواندن و نوشتن کد ساده

اهداف

باید این ساختارها را هم بفهمی و هم حداقل نسخه ساده‌شان را پیاده‌سازی کنی

Level 1

Array

String

Hash Map

Set

Level 2

Stack

Queue

Deque

Linked List

Level 3

Tree

Binary Tree

BST

Heap

Priority Queue

Level 4

Graph

Directed Graph

Weighted Graph

الگوی یادگیری

Concept

↓

Implementation

↓

Operations

↓

Complexity

↓

Use case

↓

Trade-off

↓

Problems

Mini Project — Task Queue

بساز برای مدیریت کارهای خودکار Queue یک

مثلاً:

download

process

publish

notify

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Data Structures» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

Algorithms – فصل 12

اهداف یادگیری

الگوریتم‌های جست‌وجو و مرتب‌سازی را از نظر ایده، هزینه و کاربرد توضیح بدهی.

را تشخیص بدهی DFS و BFS، recursion، sliding window، الگوهای رایج حل مسئله مانند دو اشاره‌گر.

مشخص پیاده‌سازی کنی complexity بنویسی و سپس راه‌حل را با pseudocode، برای یک مسئله.

پیش‌نیاز

مفاهیم پایه برنامه‌نویسی و توانایی خواندن و نوشتن کد ساده.

Search

Linear Search

Binary Search

Sorting

Bubble

Insertion

Selection

Merge

Quick

Heap

لازم نیست انتخاب کنی؛ هدف، فهم الگوریتمی است production همه را برای

Problem Patterns

Two Pointers

Sliding Window

Prefix Sum

Hashing

Stack

Binary Search

Recursion

BFS

DFS

Greedy

Dynamic Programming

 حل مسئله استاندارد

1. مسئله چیست؟
2. ورودی چیست؟
3. خروجی چیست؟
4. مسئله را چگونه خرد کنیم؟
5. الگوریتم چیست؟

6. Pseudocode

مثلاً:

set max to first element

for each remaining element

if current > max

set max to current

return max

7. تبدیل به کد

8. بررسی نتیجه

9. Edge Cases

 Challenge

«سومین مقدار بزرگ متمایز آرایه را پیدا کن»

pseudocode ابتدا فقط.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «Algorithms» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

 Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

 تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 13 – Discrete Mathematics

اهداف یادگیری

را با زبان خودت توضیح بدهی «Discrete Mathematics» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Discrete Mathematics» یک مثال واقعی مرتبط با

را تشخیص بدهی «Discrete Mathematics» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

مفاهیم پایه برنامه‌نویسی و توانایی خواندن و نوشتن کد ساده

اهداف

را بفهمی logic

ها را بخوانی set

را تشخیص دهی function و relation

را از دید ریاضی ببینی graph

احتمال پایه را بفهمی

مباحث

Logic

Sets

Relations

Functions

Graphs

Combinatorics

Probability basics

Boolean algebra

چرا مهم است؟

این فصل قرار نیست تو را ریاضی‌دان کند.

هدف آن تقویت دقت منطقی در الگوریتم و سیستم است.

تمرین

مدل کن Boolean را با منطق permissions یک سیستم.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Discrete Mathematics» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی.

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART IV — WEB PLATFORM

14 فصل — HTML و CSS

اهداف یادگیری

را با زبان خودت توضیح بدهی «HTML و CSS» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «HTML و CSS» یک مثال واقعی مرتبط با

را تشخیص بدهی «HTML و CSS» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

JavaScript پایه و آشنایی مقدماتی با HTML/CSS.

اهداف

بنویسی semantic HTML.

فرم دسترس‌پذیر بسازی.

بفهمی copy/paste را فراتر از CSS.

بسازی responsive design.

HTML

Semantic HTML

Forms

Inputs

Buttons

Links

Accessibility

SEO basics

Progressive enhancement

CSS

Cascade

Specificity

Inheritance

Box model

Flexbox

Grid

Positioning

Responsive design

Animations

Containment

Architecture

امتحانش کن

پایاده کن Grid و سپس Flexbox را ابتدا با layout یک

مقایسه کن:

کدام خواناتر است؟

کدام برای مسئله مناسبتر است؟

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «CSS و HTML» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

Checklist

☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.

می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم □

می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم □

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 15 – Browser Engineering

اهداف یادگیری

را با زبان خودت توضیح بدهی «Browser Engineering» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Browser Engineering» یک مثال واقعی مرتبط با

را تشخیص بدهی «Browser Engineering» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

JavaScript پایه و آشنایی مقدماتی با HTML/CSS.

اهداف

DOM را بفهمی.

event propagation را توضیح بدهی.

storage را بفهمی.

rendering را در سطح مفهومی درک کنی.

DevTools را حرفه‌ای‌تر استفاده کنی.

Rendering

HTML

↓

DOM

CSS

↓

CSSOM

↓

Render Tree

↓

Layout



Paint



Composite

مباحث

DOM

Events

Event propagation

Event delegation

localStorage

sessionStorage

Cookies

Cache

Service Workers

Web Workers

Memory

Browser security model

DevTools

Network panel

Performance panel

خطای رایج

JavaScript فقط از داخل bug جست‌وجوی

گاهی مشکل واقعی در

network

rendering

CSS

cache

state

است.

Mini Project — Offline Web App

ادامه دهد network را بدون functionality شدن بتواند بخش محدودی از load یک وب‌اپ ساده بساز که بعد از

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Browser Engineering» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 16 – HTTP و Web Protocols

اهداف یادگیری

را با زبان خودت توضیح بدهی «Web Protocols و HTTP» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Web Protocols و HTTP» یک مثال واقعی مرتبط با

را تشخیص بدهی «Web Protocols و HTTP» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

JavaScript پایه و آشنایی مقدماتی با HTML/CSS.

اهداف

request/response بدهی توضیح بدهی.

DNS، TCP، TLS و HTTP زنجیره‌ای بینی.

CORS و caching را بفهمی.

API consumption دهی انجام دهی.

زنجیره

Browser

↓

DNS

↓

TCP

↓

TLS

↓

HTTP

↓

Server

↓

Response



Browser

مباحث

HTTP/1.1

HTTP/2

HTTP/3

Headers

Cookies

Caching

Compression

CORS

Authentication

REST

WebSocket

SSE

CDN

امتحان کن

واقعی را باز کن و این‌ها را پیدا کن request یک DevTools در

Method

Status

Headers

Request body

Response body

Timing

Challenge

بساز که API client یک

داشته باشد timeout.

خطا را گزارش کند.

نامعتبر را رد کند response.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Web Protocols و HTTP» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART V — FRONTEND ENGINEERING

React — فصل 17

اهداف یادگیری

را با زبان خودت توضیح بدهی «React» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «React» یک مثال واقعی مرتبط با

را تشخیص بدهی «React» های مرتبط با trade-off خطاهای رایج و

پیش نیاز

فصل‌های پیشین React/Frontend و مباحث HTML/CSS، JavaScript، Browser

اهداف

را بفهمی component architecture

را درست تفکیک کنی state و props

را درک کنی React ذهنی lifecycle

را بفهمی rendering behavior

ها را درست استفاده کنی hook

Core

Components

Props

State

Events

Conditional Rendering

Lists

Keys

Forms

Hooks

useState

useEffect

useMemo

useCallback

useRef

useContext

Custom Hooks

اصل مهم

یاد نگیر syntax را برای حفظ React.

باید بتوانی جواب بدهی:

باید اینجا باشد؟ state چرا این

شود؟ render باید دوباره component چرا این

لازم است؟ effect چرا این

🔧 امتحانش کن

داشته باشد state بساز که component یک

منتقل کن child به parent را از state بعد

رفتار را مقایسه کن.

🐛 خطای رایج

نیست side effect برای چیزی که اصلاً effect استفاده از

🔧 Mini Project — Dashboard

بساز با dashboard یک

API

loading

error

data state

filters

reusable components

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «React» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

React Ecosystem و Next.js – فصل 18

اهداف یادگیری

را با زبان خودت توضیح بدهی «React Ecosystem و Next.js» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «React Ecosystem و Next.js» یک مثال واقعی مرتبط با

را تشخیص بدهی «React Ecosystem و Next.js» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

فصل‌های پیشین React/Frontend و مباحث HTML/CSS، JavaScript، Browser

اهداف

را بفهمی package management

را در سطح مفهومی بشناسی bundler

را در سطح کاربردی بشناسی Node.js

Next.js را به عنوان معماری وب مدرن استفاده کنی.

مباحث

npm

package.json

dependencies

modules

bundling concepts

React/Redux concepts

Node.js

Next.js

App Router

Server Components

Client Components

Routing

Data fetching

Caching

Revalidation

Server Actions

Authentication

Metadata

Deployment

Performance

تمرین

را به دو بخش تقسیم کن app یک

Server

Client

و برای هر بخش دلیل انتخاب را بنویس

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «React Ecosystem و Next.js» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 19 – Testing

اهداف یادگیری

را با زبان خودت توضیح بدهی «Testing» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Testing» یک مثال واقعی مرتبط با

را تشخیص بدهی «Testing» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

فصل‌های پیشین React/Frontend و مباحث Browser، JavaScript، HTML/CSS.

اهداف

Unit test بنویسی.

Integration test را بفهمی.

E2E test اجرا کنی.

test strategy طراحی کنی.

مدل

Unit

↓

Integration

↓

E2E

ابزارهای مسیر

Jest یا Vitest

React Testing Library

Playwright

پرسش اصلی

نه:

«چه چیزی را می‌توانم تست کنم؟»

بلکه:

«کدام رفتار باید تضمین شود؟»

امتحان کن

calculator: برای یک

جمع

تقسیم

تقسیم بر صفر

نامعتبر input

را تست کن.

خطای رایج

behavior به جای implementation detail تست

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «Testing» این فصل یک لایه از مهارت و به کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 20 – Frontend Architecture

اهداف یادگیری

را با زبان خودت توضیح بدهی «Frontend Architecture» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Frontend Architecture» یک مثال واقعی مرتبط با

را تشخیص بدهی «Frontend Architecture» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

فصل‌های پیشین React/Frontend و مباحث HTML/CSS، JavaScript، Browser

اهداف

تعریف کنی component boundary

state boundary تعریف کنی.

data layer جدا کنی.

architecture trade-off را توضیح بدهی.

مباحث

Feature-based architecture

Component boundaries

State boundaries

Data layer

API layer

Error boundaries

Design systems

Reusable components

Dependency management

Code splitting

Lazy loading

Trade-offs

Global State

vs

Local State

SSR

vs

CSR

Server Component

vs

Client Component

Challenge

ها را بنویس trade-off دو معماری پیشنهاد بده و ecommerce dashboard برای یک

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Frontend Architecture» این فصل یک لایه از مهارت و به کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 21 – Performance و Accessibility

اهداف یادگیری

را پیدا و اندازه‌گیری کنی Web در performance گلوگاه‌های رایج

را در سطح کاربردی توضیح بدهی INP و CLS و LCP.

بررسی کنی accessibility و keyboard navigation یک رابط کاربری را از نظر

پیش‌نیاز

فصل‌های پیشین React/Frontend و مباحث Browser، JavaScript، HTML/CSS.

Performance

Rendering

Bundle size

Lazy loading

Code splitting

Caching

Image optimization

Network optimization

Core Web Vitals

LCP

CLS

INP

Accessibility

Semantic HTML

ARIA

Keyboard navigation

Focus management

Screen reader support

Contrast

Accessible forms

امتحان کن

یک صفحه را

با network throttling

با keyboard only

با DevTools Performance

بررسی کن.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Performance و Accessibility» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART VI — SOFTWARE ENGINEERING

فصل 22 — Git Like an Engineer

اهداف یادگیری

را با زبان خودت توضیح بدهی «Git Like an Engineer» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Git Like an Engineer» یک مثال واقعی مرتبط با

را تشخیص بدهی «Git Like an Engineer» های مرتبط با trade-offهای رایج و

پیش‌نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

اهداف

بفهمی version control را به عنوان Git.

طراحی کنی branch workflow.

حل کنی conflict.

را تحلیل کنی history.

انجام دهی code review.

مباحث

Repository

Commit

Branch

Merge

Rebase

Fetch

Pull

Push

Stash

Reset

Revert

Cherry-pick

Reflog

Tags

GitHub

Pull Request

Issues

Actions

Secrets

Releases

Code Review

Challenge

ایجاد کن و حلش کن conflict اضافه کن، عمداً feature، بساز branch یک

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Git Like an Engineer» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 23 – Debugging و Code Quality

اهداف یادگیری

بررسی کنی reproducible debugging را با فرآیند bug یک

نقد کنی maintainability و coupling، readability، کد را از نظر

مناسب طراحی کنی verification تست و regression برای جلوگیری از

پیش نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

Debugging

Observe

↓

Reproduce

↓

Hypothesize

↓

Inspect

↓

Isolate

↓

Fix

↓

Verify

↓

Prevent regression

Code Quality

Naming

Readability

Modularity

Cohesion

Coupling

Abstraction

Separation of concerns

DRY

YAGNI

SOLID

Refactoring

Technical debt

Debugging تمرین

واقعی از یکی از پروژه‌های قبلیت انتخاب کن bug یک

بنویس:

Expected:

Actual:

Reproduction:

Root cause:

Fix:

Regression test:

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Code Quality و Debugging» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

Checklist

☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

24 فصل – Security

اهداف یادگیری

را با زبان خودت توضیح بدهی «Security» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Security» یک مثال واقعی مرتبط با

را تشخیص بدهی «Security» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

اهداف

authentication و authorization بگذاری را فرق بگذاری

attack surface را بشناسی

validate ورودی را

secrets را درست مدیریت کنی

مباحث

Authentication

Authorization

Sessions

Cookies

JWT

OAuth

XSS

CSRF

SQL Injection

SSRF

Command Injection

CORS

CSP

Secrets

TLS

Rate limiting

Least privilege

اصل

سیستم است property نیست؛ Security feature.

امتحانش کن

یک فرم را بررسی کن و بگو:

ی می تواند خطرناک باشد؟ input چه

می شود؟ validate کجا

می شود؟ encode کجا

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «Security» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 25 – Node.js و API Engineering

اهداف یادگیری

را با زبان خودت توضیح بدهی «Node.js و API Engineering» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Node.js و API Engineering» یک مثال واقعی مرتبط با

را تشخیص بدهی «Node.js و API Engineering» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

اهداف

را بفهمی backend runtime

انجام دهی API design

استاندارد داشته باشی error handling

طراحی کنی contract

Node.js

Runtime

Modules

Filesystem

HTTP

Streams

Buffers

Event loop

Processes

Environment

Package management

API

REST

RPC concepts

GraphQL

API contracts

Versioning

Pagination

Filtering

Rate limiting

Auth

Error design

Mini Project — Market API v2

فعلى خودت را به نسخه آموزشى تبديل كن API

GET /market

GET /weather

GET /news

GET /health

endpoint: برای هر

contract

validation

error response

status code

documentation

تعریف کن.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «API Engineering و Node.js» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 26 – SQL و Databases

اهداف یادگیری

مناسب را انتخاب کنی JOIN را بنویسی و SQL های اصلی query

را در سطح کاربردی توضیح بدهی ACID و transaction، index.

ساده و قابل دفاع طراحی کنی schema، برای یک برنامه

پیش نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

SQL

SELECT

WHERE

ORDER BY

GROUP BY

HAVING

JOIN

INSERT

UPDATE

DELETE

Database Engineering

Schema

Primary key

Foreign key

Index

Constraints

Normalization

Transactions

ACID

Isolation

Locking

Query planning

Replication

Consistency

NoSQL concepts

Key-value

Document

Wide-column

Graph



فرضی برای schema یک Freedom برای:

users

sources

subscriptions

deliveries

jobs

logs

طراحی کن.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «SQL و Databases» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 27 – Networking, Linux و Operating Systems

اهداف یادگیری

شبکه را توضیح بدهی request مسیر یک

process، thread، memory و concurrency درک کنی

استفاده کنی log و process، file، permission برای بررسی Linux از

پیش نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

Networking

IP

TCP

UDP

DNS

TLS

Routing

NAT

Proxy

CDN

Load Balancer

Latency

Bandwidth

Linux

Shell

Processes

Files

Permissions

Environment

Network commands

Logs

Services

Package managers

SSH

Container basics

Operating Systems

Process

Thread

Memory

Virtual memory

Filesystem

Scheduling

Synchronization

Race conditions

Deadlocks

Concurrency

امتحانش کن

Linux: در یک سیستم

پیدا کن process یک

آن را پیدا کن PID

مرتبط را پیدا کن log

دلیل اجرای آن را توضیح بده

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. «Networking، Linux و Operating Systems» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، معیار یادگیری، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 28 – Distributed Systems

اهداف یادگیری

را با زبان خودت توضیح بدهی «Distributed Systems» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Distributed Systems» یک مثال واقعی مرتبط با

را تشخیص بدهی «Distributed Systems» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

اهداف

مقیاس‌پذیری را بفهمی.

طراحی کنی failure mode

را بفهمی queue و cache

را به معماری اضافه کنی reliability

System Design

Requirements

↓

API

↓

Data Model

↓

Architecture

↓

Bottlenecks

↓

Failure

↓

Trade-offs

Level 1

Requirements

API

Data model

Level 2

Caching

Queues

Workers

Load balancing

Rate limiting

Level 3

Replication

Sharding

Consistency

Availability

Fault tolerance

Level 4

Distributed systems

Disaster recovery

Multi-region

Observability

Project — Freedom Architecture

را به صورت معماری رسم کن Freedom

Sources

↓

Scheduled Jobs

↓

Processing

↓

Selection

↓

Subscription

↓

Cloudflare

↓

Telegram / Bale



Users

:سپس برای هر مرحله

failure mode

retry

timeout

monitoring

.مشخص کن

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Distributed Systems» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

29 فصل — Reliability، CI/CD و Cloud

اهداف یادگیری

را از هم تفکیک کنی SLA، SLO و SLI.

طراحی کنی CI/CD ساده pipeline یک

را در نظر بگیری rollback و health checks، metrics، logging برای یک سرویس

پیش‌نیاز

فصل‌های قبلی Software Engineering و Web Engineering مفاهیم

Reliability

SLI

SLO

SLA

Monitoring

Alerting

Logs

Metrics

Traces

Incident response

RCA

Postmortem

CI/CD

Pipeline

Build

Test

Deploy

Environments

Secrets

Release

Rollback

Cloud

Compute

Storage

Networking

Serverless

Containers

Databases

Queues

CDN

Edge computing

تجربه فعلی تو

تو قبلاً با

GitHub Actions

Cron

Cloudflare Workers

Serverless

Bots

APIs

کار کرده‌ای.

هدف این فصل تبدیل:

«بلدم اجراش کنم»

به:

«می‌فهمم چرا این معماری مناسب است»

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Reliability، CI/CD و Cloud» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART VII — AI-ASSISTED ENGINEERING

AI Engineering تا Vibe Coding فصل 30 — از

اهداف یادگیری

را با زبان خودت توضیح بدهی «AI Engineering تا Vibe Coding مفاهیم اصلی» از

را تحلیل یا پیاده‌سازی کنی «AI Engineering تا Vibe Coding یک مثال واقعی مرتبط با» از

را تشخیص بدهی «AI Engineering تا Vibe Coding های مرتبط با» از trade-off خطاهای رایج و

پیش‌نیاز

مفاهیم مهندسی نرم‌افزار و توانایی کار با پروژه‌های واقعی

اهداف

را به ابزار واقعی مهندسی تبدیل کنی AI

انجام دهی AI روی خروجی code review.

را کنترل کنی overconfidence و hallucination.

بدهی AI مهندسی درست به context.

را تست کنی AI خروجی.

مرحله 1

AI → writes code

You → paste

این مرحله هدف نهایی نیست.

مرحله 2

You → design

AI → implementation

You → review

مرحله 3

You → architecture

You → core implementation

AI → reviewer/debugger

مرحله 4

You ↔ AI

تبدیل می‌شود به AI:

Reviewer

Debugger

Documentation assistant

Research assistant

Test generator

Adversarial reviewer

قانون طلایی

نوشته درست فرض نکن AI هیچ کدی را به دلیل اینکه

امتحان کن

بده AI را به task یک

را اجرا کن AI پاسخ اول

بعد خودت اینها را بررسی کن:

Correctness

Security

Complexity

Edge cases

Maintainability

Testing

بنویس adversarial review دوم را برای prompt سپس

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار «AI Engineering تا Vibe Coding این فصل یک لایه از مهارت» از و به‌کارگیری مفهوم در یک مسئله واقعی است debug، یادگیری، توانایی توضیح، پیاده‌سازی

Checklist

☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.

می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم □

می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم □

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART VIII — PROJECT ENGINEERING

مهندسی تبدیل کن Evidence فصل 31 — پروژه‌های شخصی را به

اهداف یادگیری

مهندسی تبدیل کن» را با زبان خودت توضیح بدهی Evidence مفاهیم اصلی «پروژه‌های شخصی را به

مهندسی تبدیل کن» را تحلیل یا پیاده‌سازی کنی Evidence یک مثال واقعی مرتبط با «پروژه‌های شخصی را به

مهندسی تبدیل کن» را تشخیص بدهی Evidence های مرتبط با «پروژه‌های شخصی را به trade-off خطاهای رایج و

پیش‌نیاز

مفاهیم مهندسی نرم‌افزار و توانایی کار با پروژه‌های واقعی.

اهداف

پروژه را فقط به‌عنوان «چیزی که ساختم» ببینی.

را مستند کنی trade-off و architecture.

پروژه را قابل دفاع کنی.

سه پروژه اصلی پیشنهادی

Freedom — پروژه 1

تمرکز:

Automation

Scheduled jobs

Serverless

APIs

Multi-platform distribution

Reliability

2 پروژه — Offline File Transfer

تمرکز:

Browser

Offline-first

Networking

Device interaction

UX

3 پروژه — Professional Frontend App

تمرکز:

React

TypeScript

Next.js

Testing

Accessibility

Performance

Architecture

repository نباید فقط.

باید داشته باشد:

README

Architecture

Why

Trade-offs

Testing

Security

Deployment

Known limitations

Future work

خلاصه فصل

مهندسی تبدیل کن» را به مسیر مهندسی اضافه Evidence این فصل یک لایه از مهارت «پروژه‌های شخصی را به و به‌کارگیری مفهوم در یک مسئله واقعی است debug، می‌کند. معیار یادگیری، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

و کار تیمی Open Source – فصل 32

اهداف یادگیری

و کار تیمی» را با زبان خودت توضیح بدهی Open Source» مفاهیم اصلی

و کار تیمی» را تحلیل یا پیاده‌سازی کنی Open Source» یک مثال واقعی مرتبط با

و کار تیمی» را تشخیص بدهی Open Source» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

مفاهیم مهندسی نرم‌افزار و توانایی کار با پروژه‌های واقعی

اهداف

بسازی issue

بدهی PR.

code review کنی.

دریافت کنی feedback.

را حل کنی conflict.

Mini Project

واقعی پیدا کن و repository open-source یک

issue را بخوان.

علت را بررسی کن.

یک تغییر کوچک بده.

تست اضافه کن.

باز کن PR.

را پاسخ بده review.

هدف این فصل:

Software Engineering is collaborative.

خلاصه فصل

و کار تیمی» را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی Open Source «این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

33 فصل — Freelance و Professional Experience

اهداف یادگیری

را تشخیص بدهی professional employment و internship، freelance تفاوت

تجربه واقعی خودت را بدون بزرگ‌نمایی مستند کنی.

برای تبدیل پروژه‌ها به تجربه حرفه‌ای برنامه‌ریزی کنی.

پیش‌نیاز

مفاهیم مهندسی نرم‌افزار و توانایی کار با پروژه‌های واقعی.

واقعیت

معرفی شوند corporate employment ارزشمندند، اما نباید به‌عنوان freelance پروژه شخصی و

رزومه باید این دسته‌ها را جدا کند:

Formal Employment

Freelance

Independent Projects

Open Source

AI-assisted Project Work

هدف

باید به مرور به freelance تجربه فعلی

Freelance

→ Internship

→ Professional Team

→ Software Engineer

تبدیل شود.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار «Freelance و Professional Experience» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، یادگیری، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART IX — TECHNICAL ENGLISH

34 فصل — English for Engineers

اهداف یادگیری

را با زبان خودت توضیح بدهی «English for Engineers» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «English for Engineers» یک مثال واقعی مرتبط با

را تشخیص بدهی «English for Engineers» های مرتبط با trade-offهای رایج و

پیش‌نیاز

مفاهیم مهندسی نرم‌افزار و توانایی کار با پروژه‌های واقعی

اهداف

Documentation بخوانی

issue بنویسی

پروژه را توضیح بدهی

design را دفاع کنی.

مصاحبه فنی انجام دهی.

Level 1

What does this code do?

Level 2

Explain your algorithm.

Level 3

Why did you choose this approach?

Level 4

What are the trade-offs?

Level 5

How would you scale this system?

Level 6

Tell me about a difficult bug you solved.

تمرین روزانه

هر جلسه:

explanation دقیقه 5-10

پاسخ کتبی 1

grammar اصلاح 1

اصطلاح فنی جدید 1

هدف:

clarity > fancy English

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «English for Engineers» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

🔴 تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART X — INTERVIEW ENGINEERING

35 فصل — Coding Interview

اهداف یادگیری

را با زبان خودت توضیح بدهی «Coding Interview» مفاهیم اصلی

را تحلیل یا پیاده‌سازی کنی «Coding Interview» یک مثال واقعی مرتبط با

را تشخیص بدهی «Coding Interview» های مرتبط با trade-off خطاهای رایج و

پیش‌نیاز

بخش عمده مسیر قبلی؛ این فصل‌ها برای تبدیل مهارت به آمادگی حرفه‌ای هستند

اهداف

مسئله را تحلیل کنی

را بگویی complexity

را توضیح بدهی solution

کد بنویسی

را بررسی کنی edge case

مراحل

Understand

→ Clarify

→ Approach

→ Complexity

→ Code

→ Test

→ Edge Cases

مباحث

Arrays

Strings

Hash Maps

Stack

Queue

Linked List

Trees

Heaps

Graphs

Recursion

Binary Search

BFS

DFS

Dynamic Programming

قانون

را حفظ نکن LeetCode.

ها را یاد بگیر Pattern.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Coding Interview» این فصل یک لایه از مهارت و به کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

 Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 36 – Technical Interview

اهداف یادگیری

مفاهیم فنی را بدون اجرای کد، دقیق و ساختاریافته توضیح بدهی.

ارائه کنی trade-off برای پاسخ فنی، تعریف، مثال، علت و

در یک گفت‌وگوی فنی از تصمیم مهندسی خود دفاع کنی.

پیش‌نیاز

بخش عمده مسیر قبلی؛ این فصل‌ها برای تبدیل مهارت به آمادگی حرفه‌ای هستند.

توضیح بدهی editor باید بتوانی بدون

چیست؟ Event Loop

چگونه کار می‌کند؟ Promise

می‌کند؟ render چگونه React

می‌کند؟ render چگونه صفحه را Browser

چه مراحل دارد؟ HTTP request

چیست؟ Cache

چیست؟ Index

چیست؟ Transaction

چیست؟ Race condition

چیست؟ Load balancer

را انتخاب کردی؟ X چرا معماری

تمرین

برای هر پاسخ:

Definition

Example

Why it matters

Trade-off

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Technical Interview» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

Checklist

☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 37 – System Design Interview

اهداف یادگیری

شروع کنی requirements را از system design.

اولیه طراحی کنی architecture و data model API.

را تحلیل کنی trade-off و failure mode bottleneck.

پیش نیاز

بخش عمده مسیر قبلی؛ این فصل‌ها برای تبدیل مهارت به آمادگی حرفه‌ای هستند.

مسائل پایه

URL shortener

Chat system

Notification system

File upload

News feed

Search

Distributed job system

الگوی پاسخ

Requirements

↓

Scale

↓

API



Data model



Architecture



Bottlenecks



Failure



Trade-offs

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «System Design Interview» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 38 – Behavioral Interview

اهداف یادگیری

تجربه‌های واقعی را به داستان‌های مصاحبه‌ای ساختاریافته تبدیل کنی.

را صادقانه و فنی توضیح بدهی bug یک شکست یا

را بیان کنی collaboration و project ownership تجربه

پیش‌نیاز

بخش عمده مسیر قبلی؛ این فصل‌ها برای تبدیل مهارت به آمادگی حرفه‌ای هستند.

داستان‌های واقعی آماده کن

difficult bug

failed project

technical decision

performance optimization

learning something difficult

collaboration

production issue

project ownership

قانون

از پروژه‌های واقعی خودت استفاده کن.

مثلاً پروژه ناموفق

Bale Live Call

را می‌توان بعداً برای توضیح

hypothesis

experimentation

failure

root cause

lesson learned

استفاده کرد.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Behavioral Interview» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

🔴 تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART XI — APPLE TRACK

فصل 39 — واقعیت Apple

اهداف یادگیری

استخراج کنی Job Description را از الگوی مهارت‌های تکرارشونده در نقش‌های مهندسی

را توضیح بدهی Software Engineer و Frontend Engineer تفاوت

تبدیل کنی evidence و skill یک نقش واقعی را به

پیش‌نیاز

و مسیر هدف skill/evidence مطالعه بخش‌های قبلی و آشنایی با مفهوم

در سپتامبر 2026 طراحی شده است Apple این فصل بر اساس الگوهای قابل مشاهده در موقعیت‌های رسمی

یک نمونه فعلی:

Front End Web Engineer, Apps and Games Engineering

internal برای performance و quality، web technology فهرست شده و روی Seattle این نقش در applications/services دارد. تأکید دارد (Apple Careers).

React، TypeScript، REST APIs، testing، Playwright، production monitoring، architecture و database work در حوزه‌های مربوط صحبت شده و برای آن 3 سال تجربه حرفه‌ای در BS/MS است. ذکر شده است (Apple Careers).

تجربه‌های چندپلتفرمی و مهندسی نرم‌افزار در مقیاس AI، روی Apple نیز Siri App Experiences در نقش فعلی (Apple Careers). بالا کار می‌کند

نتیجه

فقط Apple Engineer:

React Developer

نیست.

ترکیب مورد نیاز بیشتر شبیه این است:

Programming

+

CS

+

Software Engineering

+

Testing

+

Quality

+

Architecture

+

Systems

+

Communication

+

Specialization

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی توضیح، «Apple این فصل یک لایه از مهارت «واقعیت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 40 – Apple-Specific Skill Matrix

اهداف یادگیری

تفاوت بگذاری mastery و exposure، skill. بین

انجام بدهی gap analysis، برای یک نقش

یک مسیر اثبات مهارت تعریف کنی gap برای هر

پیش‌نیاز

و مسیر هدف skill/evidence مطالعه بخش‌های قبلی و آشنایی با مفهوم

Core Programming

JavaScript

TypeScript

یک زبان دوم در ادامه مسیر

Frontend

HTML

CSS

React

Browser

Next.js

State management

Performance

Accessibility

Testing

Engineering

Git

Code review

Debugging

Security

API design

CI/CD

CS

Big-O

Data Structures

Algorithms

OS

Networking

Databases

Systems

Distributed systems

Reliability

Observability

System design

Communication

Technical English

Documentation

Interview communication

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Apple-Specific Skill Matrix» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.

می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم □

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 41 – Swift و Apple Platforms

اهداف یادگیری

توضیح بدهی Apple-platform engineering را در مسیر SwiftUI و Swift جایگاه

را از تخصص پلتفرم جدا کنی Core engineering

شرایط مناسب اضافه کردن این شاخه به برنامه خودت را مشخص کنی

پیش نیاز

و مسیر هدف skill/evidence مطالعه بخش های قبلی و آشنایی با مفهوم

این مسیر اختیاری است

را جدی دنبال کنی Apple-platform engineering اگر بعدها تصمیم گرفتی

Swift

↓

Swift Concurrency

↓

SwiftUI

↓

UIKit concepts

↓

iOS

↓

macOS

نباید قربانی این شاخه شود Core engineering اما

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، «Swift و Apple Platforms» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توانایی توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

1 سؤال

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

2 سؤال

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

3 سؤال

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

و رزومه Portfolio – فصل 42

اهداف یادگیری

تفکیک کنی AI-assisted project experience مستقل و skill رزومه را بین

قابل دفاع توصیف کنی evidence یک پروژه را با

را حرفه‌ای و دقیق بیان کنی AI شفافیت درباره

پیش‌نیاز

و مسیر هدف skill/evidence مطالعه بخش‌های قبلی و آشنایی با مفهوم

رزومه باید بین این‌ها تفاوت بگذارد:

Independent Skills

Project Exposure

AI-assisted Experience

Freelance Experience

Professional Experience

Project Description

بد:

Built a bot with Cloudflare.

بهتر:

Built and deployed a serverless bot workflow that automated scheduled content processing and multi-platform distribution.

اما همیشه فقط چیزی را بنویس که بتوانی در مصاحبه از آن دفاع کنی.

AI Transparency

برای رزومه فعلی:

I currently describe myself as an AI-assisted "vibe coder". More than 70% of implementation across my projects has been generated or substantially assisted by AI, based on my own estimate.

و ادامه

My current independent programming ability is concentrated in basic frontend development and fundamental JavaScript. I do not claim independent mastery of every technology used in my AI-assisted projects.

این بخش باید زمانی که مهارت مستقل افزایش پیدا می‌کند به‌روزرسانی شود.

خلاصه فصل

و رزومه» را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی Portfolio» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2،

روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 43 — Application Strategy

اهداف یادگیری

تبدیل کنی evidence و requirement را به Job Description یک

را صادقانه مشخص کنی missing موارد

کنی customize رزومه را برای یک نقش مشخص

پیش نیاز

و مسیر هدف skill/evidence مطالعه بخش‌های قبلی و آشنایی با مفهوم

Apply قبل از

تطبیق بده Job Description رزومه را با

Match

Requirement

→ Evidence

نداری evidence اگر:

Missing

نه:

Invented

هدف

رزومه باید:

truthfully match

concise

evidence-based

technically defensible

باشد.

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Application Strategy» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.
- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

🧠 Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

● تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

فصل 44 – Apple-Ready Gate

اهداف یادگیری

را به شاخص‌های قابل اندازه‌گیری تبدیل کنی Apple-ready معیارهای

شکاف‌های باقی‌مانده میان سطح فعلی و هدف را تشخیص بدهی

لازم برای اثبات استقلال را مشخص کنی evidence، برای هر مهارت

پیش‌نیاز

و مسیر هدف skill/evidence مطالعه بخش‌های قبلی و آشنایی با مفهوم

این مرحله زمانی فعال است که مهارت‌ها ساخته شده باشند

Programming

☐ قوی JavaScript

☐ قوی TypeScript

☐ AI مسئله متوسط بدون

Frontend

☐ React

☐ Next.js

☐ Browser

☐ HTTP

☐ Testing

☐ Performance

☐ Accessibility

CS

☐ Big-O

☐ Data Structures

☐ Algorithms

☐ OS

☐ Networking

☐ Databases

Engineering

☐ Git

☐ Code Review

☐ Debugging

☐ Security

☐ Architecture

☐ System Design

Production

☐ CI/CD

- ☐ Cloud
- ☐ Monitoring
- ☐ Logging
- ☐ Incident response

Evidence

- ☐ پروژه‌های عمیق
- ☐ GitHub
- ☐ Portfolio
- ☐ Open Source
- ☐ Freelance/Internship/Professional experience

English

- ☐ Technical reading
 - ☐ Technical writing
 - ☐ Technical speaking
 - ☐ Interview communication
-

خلاصه فصل

را به مسیر مهندسی اضافه می‌کند. معیار یادگیری، توانایی «Apple-Ready Gate» این فصل یک لایه از مهارت و به‌کارگیری مفهوم در یک مسئله واقعی است debug، توضیح، پیاده‌سازی

✓ Checklist

- ☐ مفهوم اصلی فصل را می‌توانم توضیح بدهم.

- ☐ می‌توانم یک مثال کوچک را مستقل اجرا یا تحلیل کنم.
- ☐ می‌توانم حداقل یک خطای رایج را پیدا و اصلاح کنم.
- ☐ می‌توانم کاربرد آن را در یک پروژه واقعی توضیح بدهم.

Quiz

سؤال 1

مهم‌ترین مفهوم این فصل چیست و چرا مهم است؟

سؤال 2

یک خطای رایج در این مبحث چیست و چگونه آن را تشخیص می‌دهی؟

سؤال 3

یا سیستم اتوماسیون نام ببر API، Web App، یک کاربرد واقعی برای این مبحث در یک

کلید یادگیری

پاسخ خوب باید نشان دهد که مفهوم را می‌فهمی و می‌توانی آن را با یک مثال واقعی توضیح بدهی. در سؤال 2، روش تشخیص مهم‌تر از حفظ نام خطاست.

تمرین نهایی

یک مسئله کوچک مرتبط با فصل انتخاب کن و آن را با این قالب حل کن:

Problem:

Input:

Output:

Constraints:

Approach:

Pseudocode:

Implementation:

Test cases:

Edge cases:

What I learned:

PART XII — PROJECT PORTFOLIO FROM CURRENT EXPERIENCE

1 پروژه — FREEDOMPROJECT

ایده

communication, content distribution و automated workflows. اتوماسیون چندبخشی با تمرکز بر

اجزای مهم

Subscription Automation

از منابع پیکربندی شده configuration جمع‌آوری تقریبی 500,000

اجرای زمان‌بندی شده تقریباً هر 10 دقیقه

configuration انتخاب تصادفی تقریباً 2,100

های شخصی در صورت تنظیم configuration با merge

عمومی subscription تولید

استفاده توسط حدود 200 کاربر بر اساس اطلاعات فعلی پروژه

Distribution

Telegram

Bale

Cloudflare Workers

Scheduled jobs

Automated file publishing

تمرین مهندسی

را خودت رسم کن data flow معماری

چالش

تعریف کن failure mode برای هر مرحله

2 — Instagram Reels Distributor

تمرکز

Automation

Bot workflow

Content discovery

Automated distribution

Credentials/tokens

Cloud/serverless execution

تمرین

بنویس architecture diagram یک

3 — Telegram News MHTML

تمرکز

Browser automation

Page archiving

MHTML

Telegram/Bale distribution

Playwright/Chromium concepts

چالش

تفاوت بین

HTML

MHTML

Screenshot

PDF

را از نظر کاربردی توضیح بده

4 — Offline File Transfer

تمرکز

Browser

Caching

Offline-first

Device-to-device transfer

Connection-code pairing

Messaging-assisted pairing

پروژه ارتقایافته

نسخه جدید باید شامل

protocol واضح بودن

pairing امنیت

error handling

user feedback

test plan

باشد.

Bale/Vercel Bridge – پروژه 5

تمرکز

GitHub Actions

Remote workflow triggering

Bale bot control

Serverless bridge

Automation

چالش

چگونه بفهمد؟ bot شد، کاربر از GitHub Actions fail طراحی کن که اگر

6 پروژه — GitHub Project Manager

تمرکز

GitHub API

ZIP upload

Repository operations

Mobile-oriented workflow

Cloudflare Worker

Bot UI

تمرین

بنویس API contract های زیر operation برای

create repository

upload archive

delete files

status

7 پروژه — Market API

تمرکز

Cloudflare Workers

External data sources

JSON

CORS

API normalization

Apple Shortcuts integration

تمرین

طراحی کن Failure tolerance:

USD failed

Gold succeeded

Weather succeeded

خروجی چه باشد؟

AI Multi-Provider Bot – پروژه 8

تمرکز

AI provider routing

API integration

Bot interface

Error handling

AI access during limited connectivity

چالش

ها طراحی کن provider برای router:

Provider A

↓

429

↓

Provider B

↓

5xx

↓

Provider C

Cron Control Bot – پروژه 9

تمرکز

Cron

Scheduled automation

Remote control

Bot administration

چالش

های حساس operation برای

authentication

authorization

audit log

اضافه کن.

پروژه‌های شکست خورده

برخی پروژه‌های ناموفق باید از رزومه اصلی جدا شوند، اما از آموزش حذف نمی‌شوند.

مثلاً:

Bale Live Call

LTE experiment

آنها می‌توانند برای یادگیری استفاده شوند

Hypothesis

Experiment

Evidence

Failure

Root Cause

Lesson

یک مهندس حرفه‌ای از شکست پروژه چیزی یاد می‌گیرد؛ شکست را به موفقیت جعلی تبدیل نمی‌کند.

PART XIII — PROGRAM SCHEDULE

برنامه روزانه

سطح فعلی

دقیقه 75–90

15 min Review

25 min New concept

25 min Coding

10 min Technical English

10–15 min Test

مرحله حرفه‌ای‌تر

دقیقه 90–120

قسمت اضافه

+ Problem Solving

+ Project work

برنامه هفتگی

روز 6

یادگیری

روز 1

استراحت یا مرور سبک

Weekly Test

3–5 coding problems

2 concept questions

1 debugging task

2 English questions

سیستم ارزیابی

Understand

Implement

Debug

Explain

Apply

English

هر مورد از

0-100

ارزیابی می‌شود.

تفسیر

<60 Review Required

60-79 Developing

80-89 Acceptable

90+ Strong

Apple این نمره‌ها معیار داخلی آموزشی هستند، نه معیار رسمی.

مسیر پیشرفت

Programming

— Variables ✓

— Conditions ✓

— Loops ✓

— Arrays ●

— Objects ●

APPLE SOFTWARE ENGINEERING TRACK

└── Functions ☒

└── Async JS ☐

└── TypeScript ☐

└── React ☐

APIs ☐

Computer Science

└── Computational Thinking ☒

└── Big-O ☐

└── Data Structures ☐

└── Algorithms ☐

└── Systems ☐

Engineering

└── Git ☒

└── Testing ☐

└── Debugging ☒

└── Security ☐

└── CI/CD ☒

└── System Design ☐

English

└── Technical Reading ☒

└── Technical Writing ☒

└── Technical Speaking ☒

└── Interview English ☐

نقشه چندساله پیشنهادی

تغییر می‌کند mastery زمان دقیق بر اساس

Stage 1 — Programming Foundation

هدف:

Independent Programmer

تمرکز:

JavaScript

Problem solving

Functions

Data

Stage 2 — Frontend Foundation

هدف:

Independent Frontend Developer

تمرکز:

HTML/CSS

JS

Browser

React

TypeScript

Stage 3 — Engineering

هدف:

Frontend Engineer

تمرکز:

Testing

Debugging

Architecture

APIs

Git

Security

Performance

Stage 4 — CS

هدف:

Computer Science Competence

تمرکز:

DSA

Algorithms

Big-O

OS

Networks

Databases

Stage 5 — Systems

هدف:

Software Engineer

تمرکز:

Distributed systems

Reliability

Cloud

System design

Stage 6 — Professional

هدف:

Production Engineer

تمرکز:

Team experience

Open source

Freelance

Internship

Code review

Production systems

Stage 7 — Interview

هدف:

Interview Ready

Stage 8 — Apple

هدف:

Apple Application Ready

نکات حرفه‌ای برای این مسیر

1. تعداد تکنولوژی‌ها معیار نیست

بلد بودن:

20 frameworks

یک زبان و مهندسی آن ارزش کمتری دارد mastery بدون درک اصول، از

2. دشمن نیست AI

باعث شود AI اگر

faster prototype

better debugging

more experiments

بکنی، ابزار مفیدی است

فنی حفظ شود ownership اما باید

3. پروژه‌های فعلی تو سرمایه‌اند

از صفر شروع نشده‌اند Freedom و automation experience

کاری که باید انجام شود

experience understanding engineering depth

4. پروژه جدید بی‌دلیل نساز

هر پروژه جدید باید یک سؤال آموزشی یا یک قابلیت مهندسی روشن داشته باشد

5. شکست را ثبت کن

یک پروژه شکست‌خورده، اگر درست تحلیل شده باشد، از یک پروژه جعلی «موفق» ارزشمندتر است

سپتامبر 2026 — Apple Reality Check

این بخش متغیر است و باید قبل از هر ایلای دوباره بررسی شود

شامل نمونه‌های فعلی:

Front End Web Engineer, Apps and Games Engineering

Software Engineer (Full Stack), G&A Solutions Engineering

Software Engineer, Siri App Experiences

Reliability و AI, Quality, Developer Experience موقعیت‌هایی در حوزه

الگوی مشترک مشاهده‌شده در این نقش‌ها شامل ترکیبی از

Programming

Frontend / Software

Testing

APIs

Architecture

Quality

Production

Collaboration

است.

تأکید دارد web technology، quality و performance روی Front End Web Engineer نمونه

تا design از ownership و React/TypeScript/REST/testing/Playwright فعلی Full Stack نمونه

را شامل می‌شود production monitoring و deployment

چندپلتفرمی قرار دارد experiences و AI، privacy در تقاطع Siri App Experiences نقش

صفحات رسمی مورد استفاده برای این فصل:

<https://jobs.apple.com/en-us/details/200677054-3337/front-end-web-engineer-apps-and-games-engineering>

<https://jobs.apple.com/en-us/details/200679481/software-engineer-full-stack-g-a-solutions-engineering>

<https://jobs.apple.com/en-us/details/200664856-3337/software-engineer-siri-app-experiences>

<https://www.apple.com/careers/us/work-at-apple.html>

رسمی روز Job Description **نکته:** شرایط نقش‌ها و موقعیت‌ها تغییر می‌کنند. هیچ بخشی از این فصل نباید جای
ایلاهی را بگیرد.

APPENDIX A — MASTER CHECKLIST

Programming

☒ Variables

☐ Types

☐ Conditions

☐ Loops

☐ Arrays

☐ Objects

☐ Functions

☐ Scope

☐ Closures

☐ Async

☐ Promises

☐ Modules

☐ Error handling

TypeScript

☐ Interfaces

☐ Unions

☐ Generics

☐ Narrowing

☐ Utility types

☐ API contracts

Web

☐ HTML

☐ CSS

☐ Browser

☐ DOM

☐ HTTP

☐ CORS

☐ Cookies

☐ Cache

☐ WebSocket

☐ Service Workers

Frontend

☐ React

☐ Hooks

☐ State

☐ Architecture

☐ Next.js

☐ Performance

☐ Accessibility

☐ Testing

Software Engineering

☐ Git

☐ PR

☐ Code review

☐ Debugging

☐ Refactoring

☐ Security

☐ CI/CD

☐ Documentation

Databases

☐ SQL

☐ JOIN

☐ Index

☐ Transaction

☐ ACID

☐ Normalization

☐ Replication

Systems

☐ Networking

☐ Linux

☐ OS

☐ Processes

- ☐ Threads
- ☐ Concurrency
- ☐ Distributed systems
- ☐ System design
- ☐ Reliability
- ☐ Observability

AI

- ☐ AI-assisted coding
- ☐ Review
- ☐ Prompt engineering
- ☐ Test generation
- ☐ Adversarial review
- ☐ Agentic workflows

Career

- ☐ Projects
- ☐ GitHub
- ☐ Portfolio
- ☐ Open source
- ☐ Freelance
- ☐ Internship

☐ Professional experience

☐ Resume

☐ Interview

English

☐ Documentation

☐ GitHub writing

☐ Technical explanations

☐ Interview English

APPENDIX B — GLOSSARY

Algorithm

دستورالعمل مرحله به مرحله برای حل یک مسئله.

API

رابطی که به نرم افزارها اجازه می دهد با یکدیگر ارتباط برقرار کنند.

Array

ساختاری برای نگهداری مجموعه ای از مقادیر مرتب.

Asynchronous

کاری که لازم نیست اجرای برنامه را تا پایان عملیات متوقف کند.

Big-O

روش توصیف رشد هزینه زمانی یا حافظه ای الگوریتم.

Browser

می‌کند render را اجرا و Web application محیطی که

Cache

محلی برای نگهداری موقت داده برای دسترسی سریع‌تر

Closure

بیرونی lexical برای دسترسی به محیط function قابلیت

CI/CD

نرم‌افزار deploy و test، build فرایند خودکار

Component

واحد قابل استفاده مجدد رابط کاربری

Cron

مکانیسم اجرای زمان‌بندی شده کارها

Data Structure

روشی برای سازمان‌دهی و دسترسی به داده

Debugging

فرایند پیدا کردن و رفع علت یک رفتار اشتباه

Deployment

انتقال نرم‌افزار به محیط اجرا

Event Loop

runtime در async مکانیسم هماهنگ کردن اجرای کارهای

Function

قطعه‌ای از منطق قابل استفاده مجدد

Hash Map

key بر اساس lookup ساختاری برای

HTTP

پروتکل اصلی تبادل درخواست و پاسخ در وب.

Latency

زمان لازم برای طی شدن یک عملیات یا درخواست.

Promise

async نماینده نتیجه آینده یک عملیات.

Race Condition

شرایطی که نتیجه به ترتیب یا زمان بندی اجرای عملیات وابسته می شود.

React

component-based کتابخانه ای برای ساخت رابط های کاربری.

REST

HTTP و resource بر پایه API سبکی رایج برای طراحی.

Serverless

بر عهده توسعه دهنده نیست server infrastructure مدلی که در آن مدیریت مستقیم.

SQL

relational database زبان رایج برای کار با.

TypeScript

استاتیک type با سیستم JavaScript زبان توسعه یافته بر پایه.

WebSocket

server و client راهی برای ارتباط تعاملی و دوطرفه طولانی مدت بین.

APPENDIX C — Daily Journal

هر جلسه این قالب را تکمیل کن:

DATE:

MAIN TOPIC:

WHAT I LEARNED:

WHAT I COULD IMPLEMENT INDEPENDENTLY:

WHAT I NEEDED AI FOR:

BUG I FOUND:

HOW I DEBUGGED IT:

ENGLISH PRACTICE:

DAILY TEST SCORE:

WEAKNESS:

NEXT SESSION:

APPENDIX D — Monthly Review

هر ماه:

Independent Coding:

AI Dependency:

Problem Solving:

Frontend:

CS:

English:

Projects:

Debugging:

Testing:

Git:

سپس سه سؤال

می‌توانم انجام دهم که ماه قبل نمی‌توانستم؟ AI چه چیزی را امروز بدون

کجاست؟ AI بیشترین وابستگی من به

مهم‌ترین شکاف ماه بعد چیست؟

APPENDIX E — The Final Transformation

مسیر این کتاب در یک جمله

AI-assisted project builder



Independent programmer



Frontend developer



Frontend engineer



Software engineer



Production engineer



Professional engineer



Interview-ready engineer



Apple candidate

و مهم‌تر:

AI-generated code



AI-assisted implementation



AI-assisted engineering



Engineer using AI

را کنار بگذاری AI هدف نهایی این نیست که

بدون دانش تو تصمیم‌گیرنده نباشد AI هدف این است که

APPENDIX F — Current Starting Point

وضعیت شروع این کتاب

Phase:

JavaScript Fundamentals

Current focus:

Arrays

Objects

Loops

Functions

Basic Problem Solving

Technical English:

Basic technical explanations

Primary specialization:

Frontend Engineering

Current development mode:

AI-assisted / vibe coding

Primary long-term target:

Apple Software Engineering

اولین مأموریت

تا زمانی که این پایه تثبیت نشده

را عمیق شروع نکن React.

را جدی شروع نکن System Design.

پیشرفته نرو LeetCode.

پروژه جدید نساز GitHub فقط برای پر کردن.

ابتدا:

JavaScript Thinking Functions Async TypeScript

حرفه‌ای شو Web و Frontend سپس وارد

سخن پایانی

تو از نقطه‌ای غیرمعمول شروع می‌کنی:

آمده، اما در همان مسیر توانسته‌ای سیستم‌هایی واقعی، خودکار و گاهی مورد AI بخش بزرگی از تجربه پروژه‌های تو از استفاده بسازی.

این نه چیزی است که باید پنهان شود و نه چیزی است که باید برایش خجالت کشید.

اما از امروز یک مسئولیت جدید داری:

باید دانش تو به اندازه پروژه‌هایت رشد کند.

وقتی روزی برسد که بتوانی یک سیستم را

طراحی کنی،

هسته آن را پیاده کنی AI بدون

سرعت بگیری AI با

،تستش کنی

،کنی debug

،را مدیریت کنی failure

،آن را توضیح بدهی architecture

،هایش را دفاع کنی trade-off

،و همه اینها را به انگلیسی انجام دهی

،نیستی Vibe Coder آنوقت دیگر فقط یک

هستی Software Engineer.

در آن مرحله دیگر «نقطه شروع» نیست Apple و

یکی از مقصدهایی است که می‌توانی برایش اقدام کنی.

END